Guião · João

Investidores (fundamentalista, tendência/manada), dados/Python (BCP, calibração, KPIs), factos estilizados e pressupostos

SAD · Grupo 2 · Volatility Interruptions and Crash Contagion in Euronext Lisbon · entrega a 2 de dezembro. Os nomes vêm de `docs/CONTRATO.md`.

Resumo das tuas semanas

Semana	Entrega concreta
1 (17–24 out)	**Dados do BCP** (Python + Claude Code) e **factos estilizados com os valores-alvo** (`docs/FACTOS_ESTILIZADOS.md`), antes de qualquer calibração. Desenhar os statecharts deste guião.
2 (24–31 out)	**Fundamentalist a funcionar.** Tabela de metas (calmo e períodos de stress).
3 (31 out–7 nov)	**Chartist com rede e efeito manada.** Testar o `kpis.py` com as primeiras corridas da Margarida.
4 (7–14 nov)	**Calibração**: `compare_model.py` + afinar os investidores com a Margarida. Validação (ESMA).
5 (14–21 nov)	KPIs dos cenários (`kpis.py`). Fechar o registo de pressupostos (`docs/PRESSUPOSTOS.md`) com os valores finais.
6 (21–28 nov)	`kpis.xlsx` final para a Matilde. Relatório: investidores, dados, factos estilizados, pressupostos e resultados.

0. Como trabalhar (lê isto primeiro)

Uma vez só

1. Aceita o convite do GitHub que chega ao teu email (repositório **sad-euronext-g2**).
2. Instala o **GitHub Desktop** (desktop.github.com) e entra com a tua conta.
3. No GitHub Desktop: **File → Clone repository → sad-euronext-g2 → Clone**. Fica uma pasta no teu PC (por defeito em Documentos\GitHub\sad-euronext-g2).
4. Confirma com o grupo que **todos usam a mesma versão do AnyLogic** (Help → About). Um modelo gravado numa versão mais recente pode não abrir numa mais antiga.

Todos os dias

1. GitHub Desktop: **Fetch origin → Pull** (trazes o que os outros fizeram).
2. AnyLogic: **File → Open** e abres o teu `.alp` dentro da pasta do repositório. Não reinstalas nada: o GitHub só mantém a pasta sincronizada.
3. Trabalhas e gravas (Ctrl+S).
4. GitHub Desktop: escreves uma mensagem curta (ex.: `João: fundamentalista com statechart`) → **Commit to main → Push origin**.

Onde trabalhas

- `model/Euronext_Main.alp` é o modelo principal e **só o Miguel o edita**.
- Tu trabalhas em `sandbox/joao/`. Depois do Miguel publicar o modelo base, copia `model/Euronext_Main.alp` para a tua pasta com outro nome (ex.: `sandbox/joao/joao.alp`) e abre esse. Na tua cópia podes mexer em tudo, incluindo no Main.
- Quando a tua peça funcionar: commit + push e avisa no grupo. O Miguel abre os dois modelos no AnyLogic e, no painel **Projects**, copia o teu agente (Ctrl+C) e cola-o no modelo principal (Ctrl+V). Isto só funciona sem erros se usares os nomes do `docs/CONTRATO.md`.
- Se estragares a tua cópia: GitHub Desktop → separador **History** → botão direito num commit antigo → **Revert**. Ou voltas a copiar o modelo principal.

AnyLogic em 1 minuto

Coisa	Para que serve	Onde está
Agent type	um tipo de participante (ex.: `Fundamentalist`)	Palette → Agent → arrastar **Agent**
Population	muitos agentes do mesmo tipo dentro do Main	assistente do Agent → *Population of agents*
Parameter	valor fixo que muda de cenário para cenário (ex.: `nChartists`)	Palette → Agent → Parameter
Variable	valor que muda durante a simulação (ex.: `cash`)	Palette → Agent → Variable
Function	bloco de código que se chama pelo nome	Palette → Agent → Function
Event	código que corre num instante ou de X em X minutos	Palette → Agent → Event
Statechart	os estados de um agente e o que o faz mudar de estado	Palette → Statechart
Process Modeling Library	blocos de filas (Enter, Queue, Hold, Delay, Sink…)	Palette → Process Modeling Library
`main`	dentro de qualquer agente, `main` dá acesso ao Main (ex.: `main.stock.mid()`)	—

Como desenhar um statechart

1. Abre o agente: duplo clique no tipo (ex.: `Fundamentalist`) no painel **Projects**.
2. Palette → **Statechart** → arrasta **Statechart Entry Point**. O nome fica `statechart`; não o mudes (o Miguel chama `statechart` em todos os agentes).
3. Arrasta um **State**, encosta-lhe a ponta do Entry Point e dá-lhe o nome em **Name**.
4. Transição: na Palette faz **duplo clique em Transition** (fica em modo de desenho), clica na borda do estado de origem e depois na borda do estado de destino.
5. Clica na transição e escolhe **Triggered by**: **Rate** (acontece em média X vezes por minuto, ao acaso: "de vez em quando decide"); **Timeout** (X minutos depois de entrar no estado; com Timeout 0 o agente faz a ação e volta logo); **Message** (quando chega uma mensagem: Message type `String`, Fire transition *On particular message*, Message `"HALT"`); **Condition** (usada à saída de um Branch).
6. **Branch** (o losango na Palette Statechart) serve para decidir. Cada seta que sai dele tem uma Condition; uma tem de ser **Default** (a que se segue se nenhuma outra for verdadeira).
7. Código: cada estado tem **Entry action** (corre ao entrar) e cada transição tem **Action** (corre ao passar) e **Guard** (condição extra para a transição poder acontecer).
8. **Estado composto**: desenha um State grande e arrasta outros para dentro; dentro dele põe um **Initial State Pointer** apontado para o primeiro estado. Uma seta que sai do estado grande vale para todos os de dentro: é assim que se faz o `HALT` (de qualquer estado, o agente vai para `Halted`).
9. Testar: carrega em Run; na barra de cima da janela da simulação escolhe o agente (ex.: `fundamentalists[0]`) e vês o estado ativo a verde. Para escrever na consola: `traceln("texto " + x);`.

Este guião tem o código para colar. Onde aparece **Entry action**, **Action** ou o nome de uma função, copia o bloco cinzento para esse campo. Se o AnyLogic não aceitar `statechart.receiveMessage(msg)`, usa `statechart.fireEvent(msg)`, que faz o mesmo.

Tens o Claude Code: usa-o para tudo o que é Python. Abre-o na pasta do repositório e pede, por exemplo, “corre o download_daily.py e o target_stats.py e explica-me os resultados”, ou “o kpis.py deu este erro: …”. Os scripts já estão em `scripts/`.

1. Semana 1: perceber antes de programar

1. Abre o **Market Model** das aulas e repara em 3 coisas: (a) como o statechart muda de estado com Rate e Timeout; (b) como um agente manda uma mensagem a outro (`send(...)`) e como a transição Message reage; (c) onde se define a rede de contactos (Main → Properties → **Space and network**).
2. Lê o capítulo 0 deste guião.
3. Desenha em papel os statecharts do Fundamentalist e do Chartist (secções 2 e 4) e vê se percebes cada seta. As dúvidas vão para `sandbox/joao/duvidas.md`. Não precisas de escrever mais nenhum documento: as regras estão aqui.
4. Quando o Miguel publicar o modelo base: Pull → copia `model/Euronext_Main.alp` para `sandbox/joao/joao.alp` → abre esse.

Plano B: se o modelo base ainda não estiver pronto na quinta

Não esperes. Cria um modelo de teste teu (`sandbox/joao/teste.alp`) com um **Stock falso**, que tem os mesmos nomes do verdadeiro. Os teus agentes funcionam com ele, e quando o modelo base sair só copias os teus agentes para lá (Ctrl+C / Ctrl+V no Projects).

1. Novo modelo (time units: minutes). No Main: parâmetro `initialPrice` 0.5, variáveis `fundamentalValue` (Initial value `initialPrice`) e `nextOrderId` long 0, e uma função `broadcast(String msg)` vazia.
2. **Single agent** `stock`, do tipo `Stock`. Variáveis `lastPrice` e `ma` (Initial value `main.initialPrice`) e `halted` false.
3. Funções do Stock falso:

```
// mid (Returns double)
return lastPrice;

// order (Just action; trader Agent, side int, price double, qty int): o preço mexe um pouco no sentido da ordem
lastPrice *= 1 + side * 0.0002 * qty / 1000.0;
traceln(time() + " ordem " + (side == 1 ? "COMPRA " : "VENDA ") + qty + " @ " + price);

// cancelAll (Just action; a Agent)
// vazio
```

Event `walk` no Stock falso (Rate 1): `lastPrice *= 1 + normal(0.002, 0); ma = 0.98 * ma + 0.02 * lastPrice;`

1a. Semana 1: factos estilizados (antes de calibrar)

Os professores pediram para **definir os factos estilizados antes de calibrar**. São os padrões estatísticos que o modelo tem de reproduzir. A lista já está em `docs/FACTOS_ESTILIZADOS.md` (6 factos, cada um com a métrica e o critério de “bate”). A tua parte é preencher a coluna **Valor BCP** com o `target_stats.py` (secção 1b) e fazer commit **antes** de a Margarida correr a primeira calibração. Depois disso a lista não muda, porque mudá-la a meio seria afinar a meta ao resultado.

Os professores também avisaram: validamos estatísticas de preço e volume, mas **não dizemos que o modelo reproduz crashes reais** (BES, março 2020). Os choques são estilizados, “inspirados em”, e servem para comparar regras, não para recriar a história.

1b. Semana 1: dados do BCP (Python)

1. Instala o Python (python.org/downloads; no instalador marca **"Add python.exe to PATH"**). Depois, na pasta do repositório: `pip install -r scripts/requirements.txt` (ou pede ao Claude Code para o fazer).
2. Preços diários desde 2010: `python scripts/download_daily.py` → `data/raw/bcp_daily.csv`.
3. Metas do período calmo: `python scripts/target_stats.py` → `data/processed/target_stats_2023-01-01_2024-12-31.xlsx`.
4. Períodos de stress: `python scripts/target_stats.py 2014-06-01 2014-09-30` (BES) e `python scripts/target_stats.py 2020-02-15 2020-04-30` (COVID).
5. Dados de 1 minuto: `python scripts/archive_intraday.py`. **Corre isto 2 vezes por semana até dezembro** (ex.: segunda e quinta), porque o Yahoo só guarda ~7 dias. Põe um alarme. Cada corrida acrescenta ao mesmo ficheiro sem duplicar.
6. Commit + push de `data/` (mensagem `João: dados BCP`).

Estatística (target_stats)	O que quer dizer
volatilidade_diaria	desvio-padrão dos retornos diários: o "tamanho normal" de um dia
curtose_excesso	caudas pesadas: 0 seria normal; ações reais têm muito mais (dias extremos mais frequentes)
pct_dias_var_maior_3pct	% de dias com subida ou descida > 3%: é o que faz disparar o colar dinâmico
autocorr_abs_retornos_lag1	clustering de volatilidade: dias agitados vêm seguidos
pior_dia / melhor_dia	extremos; servem para comparar com o choque do cenário BES

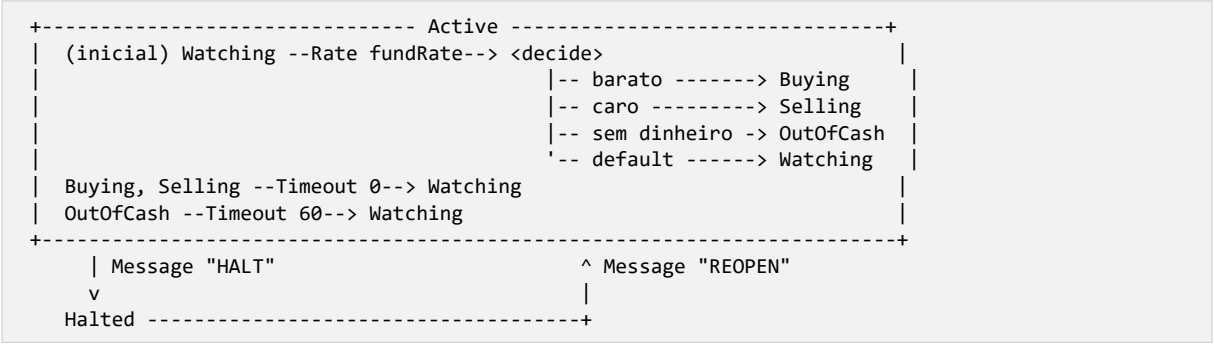
Semana 2: tabela de metas

Junta num Excel `data/processed/metas.xlsx` três colunas (Calmo 2023–24, BES 2014, COVID 2020) com as linhas do `target_stats` e a queda máxima de cada período (do pico ao mínimo). Pede ao Claude Code um script que faça isto a partir do `bcp_daily.csv`. Vai para o relatório como "factos estilizados".

2. Fundamentalist: compra barato, vende caro

Cada fundamentalista tem uma estimativa do valor da ação: o valor fundamental mais um erro pessoal. Se o preço está abaixo, compra; se está acima, vende; durante uma reserva, espera.

1. No Main da tua cópia, parâmetros: `nFundamentalists` int 40, `fundRate` 0.2, `fundBand` 0.01.
2. Palette → Agent → **Agent** → **Population of agents** → type `Fundamentalist`, population `fundamentalists`, Initial number `nFundamentalists`.
3. Variáveis do Fundamentalist: `cash` double 5000, `position` int 10000, `bias` double com Initial value `normal(0.02, 0)`.
4. Função **value** (Returns value, double): `return main.fundamentalValue * (1 + bias);`
5. Desenha o statechart:



Transição	Triggered by	Condition / Guard
Watching → decide	Rate `main.fundRate`	—
decide → Buying	Condition	`main.stock.mid() < value() * (1 - main.fundBand) && cash > main.stock.mid() * 100`
decide → Selling	Condition	`main.stock.mid() > value() * (1 + main.fundBand) && position >= 100`
decide → OutOfCash	Condition	`cash < main.stock.mid() * 100 && position < 100`
decide → Watching	Default	—
Buying → Watching, Selling → Watching	Timeout 0	—
OutOfCash → Watching	Timeout 60	volta a tentar uma hora depois
Active → Halted	Message `"HALT"`	—
Halted → Active	Message `"REOPEN"`	entra outra vez em Watching (Initial State Pointer)

Buying → Entry action:

```
double m = main.stock.mid();
double p = Math.min(value(), m * 1.002);                          // paga no máximo o que acha justo
int q = (int) Math.min(cash / p, 100 + 20000 * (value() / m - 1));    // compra mais quanto mais barato estiver
if (q >= 100) main.stock.order(this, 1, p, q);
```

Selling → Entry action:

```
double m = main.stock.mid();
double p = Math.max(value(), m * 0.998);
int q = (int) Math.min(position, 100 + 20000 * (m / value() - 1));
if (q >= 100) main.stock.order(this, -1, p, q);
```

Na tua cópia, no Main, descomenta as linhas do Fundamentalist nas funções `onFill` e `broadcast` (o Miguel faz o mesmo no principal).

Teste: corre; o preço deve oscilar à volta do valor fundamental. Escolhe `fundamentalists[0]` na barra de cima e vê o statechart a mudar. Para ver se ele reage, muda o `initialPrice` para 0.45 com o `fundamentalValue` a 0.50: o preço deve subir. Commit `João: fundamentalista` e avisa.

3. Noise trader (semana 2)

O Miguel já fez o NoiseTrader com um Event. Fica assim: os noise traders continuam a enviar ordens durante a reserva (essas ordens esperam pelo leilão, como na realidade), por isso não precisam de statechart. A tua parte é afinar `noiseRate` e `noiseSigma` na calibração (secção 5).

4. Chartist (tendência) com efeito manada (semana 3)

O chartist compra se o preço está acima da média móvel e vende se está abaixo. Quando vende, avisa os vizinhos na rede (`SELL_SIGNAL`). Se receber avisos de vizinhos suficientes, entra em pânico e vende muito: é a manada, como no Market Model.

1. Parâmetros no Main: `nChartists` int 30, `chartRate` 0.2, `trendThreshold` 0.005, `herdThreshold` int 2, `panicDuration` 10, `shockPanicProb` 0.3, `shockSize` -0.10.
2. População `chartists` (type `Chartist`, Initial number `nChartists`). Variáveis: `cash` 5000, `position` 10000, `sellSignals` int 0.
3. Rede: Main → Properties → **Space and network** → Network type **Small world**, Connections per agent **6**, Neighbor link probability 0.95. A rede liga todos os agentes do Main; por isso o código confirma que o vizinho é um Chartist.

4 Chartist → Properties → **Communication** (ou *Agent actions*) → **On message received**: `statechart.receiveMessage(msg);` Se já houver a opção *Forward message to: statechart* marcada, não escrevas nada, porque senão a mensagem chega duas vezes.

5 Event `forget` (Cyclic, 30 min): `sellSignals = 0;` (esquece avisos antigos).

6 Função **signalNeighbours** (Just action):

```
for (Agent n : getConnections())
    if (n instanceof Chartist) send("SELL_SIGNAL", n);
```

```
 +---------------------------------- Active -------------------------------------+
 |  Neutral --Rate chartRate--> <decide> --subida--> TrendBuy  --T0--> Neutral     |
 |                                 |-----descida--> TrendSell --T0--> Neutral     |
 |                                 '-----default--> Neutral                        |
 |  Neutral --"SELL_SIGNAL" (sellSignals++)--> <herd> --vizinhos>=limite--> Panic  |
 |                                              '------default----------> Neutral |
 |  Neutral --"SHOCK" (guard: prob.)-------------------------------------> Panic |
 |  Panic --Timeout panicDuration (sellSignals = 0)--> Neutral                    |
 +--------------------------------------------------------------------------------+
        "HALT" --> Halted --"REOPEN"--> Active
```

Transição	Triggered by	Condition / Guard / Action
Neutral → decide	Rate `main.chartRate`	—
decide → TrendBuy	Condition	`main.stock.lastPrice > main.stock.ma * (1 + main.trendThreshold) && cash > main.stock.mid() * 100`
decide → TrendSell	Condition	`main.stock.lastPrice < main.stock.ma * (1 - main.trendThreshold) && position >= 100`
decide → Neutral	Default	—
TrendBuy/TrendSell → Neutral	Timeout 0	—
Neutral → herd	Message `"SELL_SIGNAL"`	Action: `sellSignals++;`
herd → Panic	Condition	`sellSignals >= main.herdThreshold && position > 0`
herd → Neutral	Default	—
Neutral → Panic	Message `"SHOCK"`	Guard: `main.shockSize < 0 && randomTrue(main.shockPanicProb)`
Panic → Neutral	Timeout `main.panicDuration`	Action: `sellSignals = 0;`
Active → Halted / Halted → Active	Message `"HALT"` / `"REOPEN"`	—

```
// TrendBuy -> Entry action
double p = main.stock.mid() * 1.003;
int q = (int) Math.min(cash / p, 500);
if (q >= 100) main.stock.order(this, 1, p, q);

// TrendSell -> Entry action
main.stock.order(this, -1, main.stock.mid() * 0.997, Math.min(position, 500));
signalNeighbours();

// Panic -> Entry action (vende muito e abaixo do mercado)
if (position > 0) main.stock.order(this, -1, main.stock.mid() * 0.99, Math.min(position, 2000));
signalNeighbours();
```

A média móvel `main.stock.ma` é atualizada pelo Miguel a cada minuto. **Teste:** `nChartists = 60` e `herdThreshold = 1` devem dar quedas fortes; com `collarSet = 2` essas quedas devem abrir

reservas. Commit e avisa.

5. Calibração com a Margarida (semana 4)

1. A Margarida corre o cenário baseline no AnyLogic (10 replicações). Os resultados ficam em `model/output/precos.csv`.
2. Tu corres `python scripts/compare_model.py baseline`. O script mostra o BCP real vs. o modelo e se cada estatística **bate** (a menos de 25%). No modelo, um dia tem 510 minutos e o último preço de cada bloco conta como fecho.
3. Se não bater, escolhem juntos o que mudar (tabela abaixo), ela corre outra vez e tu comparas outra vez. Muda uma coisa de cada vez e anota cada tentativa em `data/processed/calibracao_log.xlsx` (data, parâmetros, resultados): vai para o relatório.

Se no modelo…	Experimenta…
a volatilidade diária é baixa demais	subir `noiseSigma`, subir `nChartists`, subir `fundBand`
a volatilidade é alta demais	o contrário; subir `nFundamentalists`
a curtose (caudas pesadas) é baixa demais	subir `nChartists`, descer `herdThreshold`, subir `panicDuration`
há pouco clustering de volatilidade	subir `chartRate`, subir `panicDuration`
há dias com variação > 3% a mais	subir `nFundamentalists` ou `fundRate`
o preço afasta-se do valor e não volta	subir `nFundamentalists`, `fundRate`

Validação (semana 4)

- **ESMA:** a Margarida preenche a tabela da ESMA (efeitos das interrupções). No cenário 2 com colares, na folha **halts** do `kpis.xlsx`, `vol_depois` tem de ser menor que `vol_antes` e `spread_depois` maior que `spread_antes`.
- **Choque:** a queda máxima no cenário de choque deve ser **plausível** (da mesma ordem de grandeza dos piores dias do BCP na tabela de metas). Não é um teste de reprodução do BES: no relatório escreve “inspirado em”.

6. KPIs dos cenários (semanas 3–6)

Sempre que a Margarida acabar um lote de corridas no AnyLogic, corres `python scripts/kpis.py`. Cria `data/processed/kpis.xlsx` com 3 folhas: **halts** (uma linha por interrupção), **runs** (uma por corrida) e **resumo** (média e intervalo de confiança de 95% por cenário).

KPI da proposta	Onde está no kpis.xlsx
Volatilidade antes vs. depois (2, 5, 10 min)	halts: `vol_antes_w`, `vol_depois_w`
Spread antes vs. depois	halts: `spread_antes_w`, `spread_depois_w`
Gap entre o preço do leilão e o preço antes	halts: `gap_leilao`
Queda máxima (maximum drawdown)	runs/resumo: `queda_maxima`
Volume e tempo sem negociação contínua	runs/resumo: `volume_total`, `tempo_em_interrupcao`
Tempo para recuperar metade da queda	runs/resumo: `min_recuperar_metade`
Interrupções repetidas	runs/resumo: `n_interrupcoes`
Curtose dos retornos	runs/resumo: `curtose_ret_1min`

Na semana 6 corres o `kpis.py` sobre as corridas finais e dás o `kpis.xlsx` à **Matilde**, que faz os gráficos.

7. Registo de pressupostos (semanas 1–5)

Os professores avisaram que os dados do livro de ordens e do comportamento dos investidores são escassos, e pediram para **documentar os pressupostos**. Estão em `docs/PRESSUPOSTOS.md`: cada linha diz o que assumimos, porquê e como o testamos. És tu que manténs este ficheiro:

- Sempre que alguém do grupo inventar um número (ex.: `fundBand` 0.01), acrescenta-se uma linha.
- Depois da calibração, atualizas a coluna **Valor** com os valores finais.
- Na semana 5 marcas, para cada pressuposto, se entrou na análise de sensibilidade da Margarida.

8. Semana 6: relatório

- Investidores: os statecharts e as regras deste guião (fundamentalista, chartist e manada).
- Dados, factos estilizados e pressupostos: as tabelas de `docs/FACTOS_ESTILIZADOS.md` e `docs/PRESSUPOSTOS.md`.
- Resultados: as tabelas do resumo do `kpis.xlsx`, com médias e intervalos de confiança.